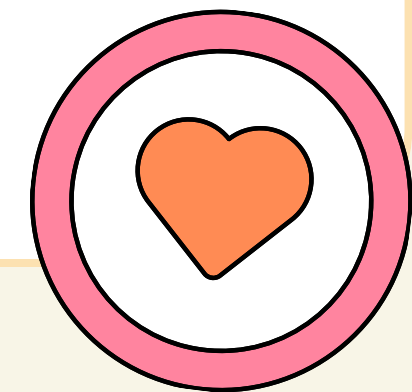


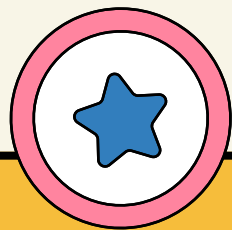
517321 PRINCIPLES OF PROGRAMMING LANGUAGES

RUST TUTORIAL PRESENTATION

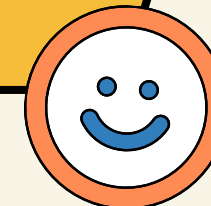
กลุ่มที่ 9 - Enums & Pattern Matching

1. วุฒิชัย หลักเพชร- Concept + Short Code Illustration
2. ศุภิสรา สอนดี - Detailed Code + Live Demo
3. จารุเดช พินิตักดา - Rust vs Other Language + PPL Analysis
4. ชัชชา เกาเสน - Exercises + Common Mistakes + Challenge





Agenga - โครงสร้างการนำเสนอ(รวม ~ 20 นาที)



แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

01

MEMBER 1

CONCEPT + SHORT CODE
ILLUSTRATION

แนวคิดหลักของหัวข้อ
พร้อมโค้ดตัวอย่างสั้น ๆ

~5 นาที

02

MEMBER 2

DETAILED CODE +
LIVE DEMO

อธิบายโค้ดเชิงลึกและ
สาธิตการทำงานจริง

~5 นาที

03

MEMBER 3

RUST VS OTHER
LANGUAGE + PPL
ANALYSIS

เปรียบเทียบภาษา
และวิเคราะห์เชิง
PROGRAMMING
LANGUAGE

~5 นาที

04

MEMBER 4

EXERCISES + COMMON
MISTAKES + CHALLENGE

ข้อผิดพลาดที่พบบ่อย และ
โจทย์ท้าทายผู้ฟัง

~5 นาที

PART 1

CONCEPT & SHORT CODE ILLUSTRATION



การประกาศ ENUM พื้นฐาน (DEFINING ENUMS) คืออะไร



ความหมาย

Custom Type ใน Rust ที่กำหนดค่าที่เป็นไปได้หลายรูปแบบ โดยแต่ละ Variant สามารถเก็บข้อมูลในรูปแบบที่ต่างกัน

เหตุผลที่ออกแบบ

เพื่อรวบรวมข้อมูลที่เกี่ยวข้องให้อยู่ด้วยกันอย่างเป็นระเบียบ และเพิ่มความปลอดภัยด้วยการเช็คข้อผิดพลาดตั้งแต่ตอน Compile time

ปัญหาที่แก้ไข

ตัดปัญหาการใช้ Magic Number (เช่น ใช้ -1 หรือ 0 แทนสถานะ) ซึ่งทำให้โค้ดอ่านยากและเสี่ยงต่อการตีความผิด

จุดสำคัญ (3 รูปแบบ Variants)

- Unit Variant: แบบไม่มีข้อมูลภายใน
- Tuple Variant: แบบเก็บข้อมูลเรียงตามลำดับ
- Struct Variant: แบบเก็บข้อมูลเป็นฟิลด์ระบุชื่อชัดเจน

```
// การสร้าง Enum ที่เก็บข้อมูลได้หลายรูปแบบ
enum Message {
    Quit, // Unit-like: ไม่มีข้อมูลข้างใน
    Move { x: i32, y: i32 }, // Struct-like: เก็บพิกัด x, y
    Write(String), // Tuple-like: เก็บข้อความ
}

fn main() {
    let msg1 = Message::Write(String::from("Hello PPL"));
    let msg2 = Message::Move { x: 10, y: 20 };

    println!("{:?}", msg1);
    println!("{:?}", msg2);
}
```

```
Write("Hello PPL")
Move { x: 10, y: 20 }
```

PATTERN MATCHING (MATCH) คืออะไร



ความหมาย

คำสั่งควบคุมทิศทางโปรแกรม คล้าย switch/case แต่มีความสามารถพิเศษในการแกะข้อมูลที่ซ่อนอยู่ใน Enum ออกมาใช้งาน

เหตุผลที่ออกแบบ

เพื่อตรวจสอบเงื่อนไขและเข้าถึงข้อมูลภายใน Enum ได้อย่างสะดวก พร้อมทั้งยกระดับความปลอดภัยในระดับภาษา

ปัญหาที่แก้ไข

ตัดปัญหาการลืมจัดการบางกรณีของข้อมูล (Uncovered Cases) ซึ่งมักทำให้โปรแกรมพัง โดยบังคับให้เขียนครอบคลุมทุกกรณี

```
enum Coin {
  Penny,
  Quarter(String), // เก็บชื่อรัฐของเหรียญ Quarter
}

fn value_in_cents(coin: Coin) -> u8 {
  match coin {
    Coin::Penny => 1,
    // Pattern Matching ดึงค่า String ออกมาใส่ตัวแปร state
    Coin::Quarter(state) => {
      println!("State quarter from {}", state);
      25
    }
  }
}

fn main() {
  // สร้างตัวอย่างเหรียญเพื่อทดสอบ
  let coin1 = Coin::Penny;
  let coin2 = Coin::Quarter(String::from("Alaska"));

  value_in_cents(coin1);
  value_in_cents(coin2);
}
```



State quarter from Alaska!

จุดสำคัญ

- Exhaustiveness: ต้องครอบคลุมทุก Variant เสมอ หากไม่ครบ Compiler จะแจ้ง Error ทันที
- Destructuring: ดึงข้อมูลที่ซ่อนอยู่ใน Variant ออกมาเก็บในตัวแปรเพื่อนำไปใช้งานต่อได้ทันที

การแทนที่ NULL ด้วย OPTION คืออะไร



ความหมาย

Enum พิเศษใน Standard Library (`Option<T>`) ที่ใช้สื่อว่าข้อมูลนั้น "มีค่า (Some)" หรือ "ไม่มีค่า (None)" แทนการใช้ Null แบบเดิม

เหตุผลที่ออกแบบ

เพื่อสร้างความปลอดภัยจากค่า Null (Null-safety) และป้องกัน Runtime Error จากการเข้าถึงข้อมูลว่างเปล่า

ปัญหาที่แก้ไข

ตัดปัญหา Null Pointer Exception โดยภาษาไม่อนุญาตให้นำค่าใน `Option<T>` ไปใช้งานตรง ๆ ทันที

```
// Rust code
// Option Enum ถูก Build-in มาในภาษา หน้าตาเป็นแบบนี้:
// enum Option<T> { Some(T), None, }

fn main() {
    let some_number: Option<i32> = Some(5);
    let absent_number: Option<i32> = None;

    match some_number {
        Some(val) => println!("มีตัวเลขคือ: {}", val),
        None => println!("ไม่มีข้อมูล (คล้าย Null แต่ปลอดภัย)",)
    }
}
```

มีตัวเลขคือ: 5

จุดสำคัญ

บังคับจัดการค่าว่าง: ต้องใช้ `match` เพื่อแกะค่า และบังคับให้จัดการกรณี `None` เสมอ ทำให้ปลอดภัยขึ้นอย่างมาก

การจัดการ ERROR ด้วย RESULT<T, E> คืออะไร



ความหมาย

Enum พิเศษ (Result<T, E>) สำหรับฟังก์ชันที่อาจเกิดข้อผิดพลาด โดยจะคืนค่า Ok(ข้อมูล) เมื่อสำเร็จ และ Err(ข้อผิดพลาด) เมื่อล้มเหลว

เหตุผลที่ออกแบบ

เพื่อให้การจัดการ Error เป็นไปอย่างชัดเจนและปลอดภัยในระดับภาษา โดยไม่ต้องพึ่งพาระบบ try/catch แบบภาษา OOP ทั่วไป

ปัญหาที่แก้ไข

ตัดปัญหาโปรแกรมหยุดทำงานกะทันหัน (Crash) จากข้อผิดพลาด (เช่น แปลงสตริงเป็นตัวเลขไม่สำเร็จ) โดยบังคับให้มีการตรวจสอบผลลัพธ์ก่อนทำงานต่อ

จุดสำคัญ

ประกอบด้วย 2 Variant หลัก คือ Ok(T) (สำเร็จ) และ Err(E) (ล้มเหลว) ซึ่งต้องนำไปใช้ร่วมกับ match เพื่อจัดการทั้งสองกรณีอย่างปลอดภัย

```
// Result Enum ถูก Build-in มาในภาษา หน้าตาเป็นแบบนี้:  
// enum Result<T, E> { Ok(T), Err(E), }
```

```
fn main() {  
    // การแปลง String เป็นตัวเลข อาจเกิด Error ได้  
    let parse_result: Result<i32, _> = "100a".parse();  
  
    match parse_result {  
        Ok(number) => println!("แปลงสำเร็จ ได้เลข: {}", number),  
        Err(e) => println!("แปลงไม่สำเร็จ เกิดข้อผิดพลาด: {}", e),  
    }  
}
```



แปลงไม่สำเร็จ เกิดข้อผิดพลาด: invalid digit found in string

CONCISE CONTROL FLOW ด้วย IF LET คืออะไร



ความหมาย

Syntax Sugar ที่ช่วยย่อรูปคำสั่ง match สำหรับกรณีที่เราสนใจจัดการ "เพียงกรณีเดียว" จาก Enum ที่มีหลายกรณี

เหตุผลที่ออกแบบ

เพื่อลดความซ้ำซ้อนและความยาวของโค้ด (Boilerplate) เมื่อต้องการตรวจสอบและทำงานกับ Variant เพียงตัวเดียว

ปัญหาที่แก้ไข

ตัดปัญหาโค้ดที่ยาวเกินความจำเป็นจากการเขียน match เต็มรูปแบบ แล้วต้องมาคอยเขียน `_ => ()` เพื่อดักกรณีอื่น ๆ ที่ไม่ได้ใช้งาน

จุดสำคัญ

- เป็นทางเลือกที่กระชับกว่า match สำหรับดักแค่ 1 กรณี
- ยังคงความสามารถในการแกะข้อมูลข้างใน (เช่น ดึงค่าจาก Some) ออกมาใช้งานต่อได้ทันทีเหมือนเดิม

```
// Rust code
fn main() {
    let config_max = Some(3u8);

    // แบบที่ 1: ใช้ match (ต้องเขียน _ => () เพื่อดักกรณีที่เหลือ)
    match config_max {
        Some(max) => println!("The maximum is configured to be {}", max),
        _ => (),
    }

    // แบบที่ 2: ใช้ if let (กระชับกว่า อ่านง่ายกว่า)
    if let Some(max) = config_max {
        println!("The maximum is configured to be {}", max);
    }
}
```

```
The maximum is configured to be 3
The maximum is configured to be 3
```

PART 2

DETAILED CODE & LIVE DEMO



DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

ตรวจเกรดจากคะแนน 0-100 ของนักศึกษาในระบบวิชา OS



ค้นหารายชื่อ
ซึ่งอาจไม่มีในระบบ



ตรวจคะแนน ตัดเกรด
คะแนนอาจจะผิดปกติ



รายงานผล
แสดงผลตามเงื่อนไขที่กำหนด

DETAILED CODE +LIVE DEMO

[1] get_grade()

```
use std::collections::HashMap;
// [1] แปลงคะแนนเป็นเกรด
fn get_grade(score: i32) -> Option<char> {
    match score {
        80..=100 => Some('A'),
        70..=79  => Some('B'),
        60..=69  => Some('C'),
        0..=59   => Some('D'),
        _        => None, // คะแนนติดลบหรือเกินช่วง
    }
}
```

DEMO 1 - Grade_Checker

#1 แปลงคะแนนเป็นเกรด

เป็น ENUM มาตรฐานของ RUST
คืนค่าเกรดออกไปตามเกณฑ์ที่กำหนด
โดยใช้ MATCH เลือกทางตามช่วง

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[1] get_grade()

```
use std::collections::HashMap;
// [1] แปลงคะแนนเป็นเกรด
fn get_grade(score: i32) -> Option<char> {
    match score {
        80..=100 => Some('A'),
        70..=79  => Some('B'),
        60..=69  => Some('C'),
        0..=59   => Some('D'),
        _        => None, // คะแนนติดลบหรือเกินช่วง
    }
}
```

คืน OPTION<CHAR>

ผลลัพธ์อาจจะมีเกรดหรือไม่มี

DETAILED CODE +LIVE DEMO

[1] get_grade()

```
use std::collections::HashMap;
// [1] แปลงคะแนนเป็นเกรด
✓ fn get_grade(score: i32) -> Option<char> {
    match score {
        80..=100 => Some('A'),
        70..=79  => Some('B'),
        60..=69  => Some('C'),
        0..=59   => Some('D'),
        _       => None, // คะแนนติดลบหรือเกินช่วง
    }
}
```

DEMO 1 - Grade_Checker

อยู่ในช่วง → **SOME**(เกรด)

MATCH ตามช่วงคะแนน

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[1] get_grade()

```
use std::collections::HashMap;
// [1] แปลงคะแนนเป็นเกรด
fn get_grade(score: i32) -> Option<char> {
    match score {
        80..=100 => Some('A'),
        70..=79  => Some('B'),
        60..=69  => Some('C'),
        0..=59   => Some('D'),
        _        => None, // คะแนนติดลบหรือเกินช่วง
    }
}
```

นอกช่วง 0 - 100 → **NONE**

เช่น 150 หรือ -5 ตัดเกรดไม่ได้

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[2] report()

#2 รายงานผลคะแนนโดยตรวจสอบตามเงื่อนไข

```
fn report(db: &HashMap<&str, i32>, name: &str) {  
    // ขั้นที่ 1: ค้นหาคะแนนจากรายชื่อนักศึกษา  
    match db.get(name) {  
        None => println!("{:<11} : Error! Not found", name), //ไม่พบชื่อ = ไม่มีเกรด (none)  
        //ขั้นที่ 2 : พบรายชื่อนักเรียนตรวจสอบเกรดที่ได้  
        Some(&score) => { match get_grade(score) {  
            Some(g) => println!("{:<11} : {} points [grade {}]", name, score, g),  
            None => println!("{:<11} : Warning! {} points --> Incorrect scores", name, score),  
        }  
    }  
}
```

DETAILED CODE + LIVE DEMO

DEMO 1 - Grade_Checker

[2] report()

#2 รายงานผลคะแนนโดยตรวจสอบตามเงื่อนไข

ขั้นที่ 1 : มีชื่อในระบบไหม?

ไม่มี → **NONE** → “ไม่เจอชื่อในระบบ”

```

fn report(db: &HashMap<&str, i32>, name: &str) {
    // ขั้นที่ 1: ค้นหาคะแนนจากรายชื่อนักศึกษา
    match db.get(name) {
        None => println!("{:<11} : Error! Not found", name), //ไม่พบชื่อ = ไม่มีเกรด (none)
        //ขั้นที่ 2 : พบรายชื่อดูตรวจสอบเกรดที่ได้
        Some(&score) => { match get_grade(score) {
            Some(g) => println!("{:<11} : {} points [grade {}]", name, score, g),
            None => println!("{:<11} : Warning! {} points --> Incorrect scores", name, score),
        }}
    }
}

```

DETAILED CODE + LIVE DEMO

[2] report()

#2 รายงานผลคะแนนโดยตรวจสอบตามเงื่อนไข

DEMO 1 - Grade_Checker

ขั้นที่ 1 : มีชื่อในระบบไหม?

มี → **SOME** → ส่งคะแนนไปตัดเกรด

```
fn report(db: &HashMap<&str, i32>, name: &str) {
    // ขั้นที่ 1: ค้นหาคะแนนจากรายชื่อนักศึกษา
    match db.get(name) {
        None => println!("{:<11} : Error! Not found", name), //ไม่พบชื่อ = ไม่มีเกรด (none)
        //ขั้นที่ 2 : พบรายชื่อนำมาตรวจสอบเกรดที่ได้
        Some(&score) => { match get_grade(score) {
            Some(g) => println!("{:<11} : {} points [grade {}]", name, score, g),
            None => println!("{:<11} : Warning! {} points --> Incorrect scores", name, score),
        }}
    }
}
```

DETAILED CODE + LIVE DEMO

[2] report()

#2 รายงานผลคะแนนโดยตรวจสอบตามเงื่อนไข

```

fn report(db: &HashMap<&str, i32>, name: &str) {
    // ขั้นที่ 1: ค้นหาคะแนนจากรายชื่อนักศึกษา
    match db.get(name) {
        None => println!("{:<11} : Error! Not found", name), //ไม่พบชื่อ = ไม่มีเกรด (none)
        //ขั้นที่ 2 : พบรายชื่อบริการสอบเกรดที่ได้
        Some(&score) => { match get_grade(score) {
            Some(g) => println!("{:<11} : {} points [grade {}]", name, score, g),
            None => println!("{:<11} : Warning! {} points --> Incorrect scores", name, score),
        }
    }
}

```

การจัดรูปแบบข้อความ
ให้แสดงผลกว้างอย่างน้อย 11 ตัวอักษร และชิดซ้าย

DEMO 1 - Grade_Checker

ขั้นที่ 2 : ตัดเกรดได้ไหม?

คะแนนถูก แสดงคะแนนพร้อมเกรด

คะแนนผิด แจ้งผลออกหน้าจอ

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[3] main()

```
fn main() {  
    println!("==== Grade Checker =====");  
    //รายชื่อที่เก็บไว้พร้อมคะแนนบุคคล  
    let db = HashMap::from([  
        ("Sanruethai", 85),  
        ("Sakaodeuan", 50),  
        ("Pancheewa", 67),  
        ("Thaenrak", 150),  
    ]);  
    //รายชื่อที่จะทำการค้นหา  
    let names = ["Sanruethai", "Sakaodeuan", "Pancheewa", "Thaenrak", "Mether"];  
    for name in names {  
        report(&db, name); //รายงานผล  
    }  
}
```

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[3] main()

```
fn main() {  
    println!("==== Grade Checker =====");  
    //รายชื่อที่ได้ไปพร้อมคะแนนรายบุคคล  
    let db = HashMap::from([  
        ("Sanruethai", 85),  
        ("Sakaodeuan", 50),  
        ("Pancheewa", 67),  
        ("Thaenrak", 150),  
    ]);  
    //รายชื่อที่จะทำการค้นหา  
    let names = ["Sanruethai", "Sakaodeuan", "Pancheewa", "Thaenrak", "Mether"];  
    for name in names {  
        report(&db, name); //รายงานผล  
    }  
}
```

ข้อมูลนักศึกษา
เก็บรายชื่อและคะแนน

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[3] main()

```
fn main() {  
    println!("===== Grade Checker =====");  
    //รายชื่อที่เก็บไว้พร้อมคะแนนบุคคล  
    let db = HashMap::from([  
        ("Sanruethai", 85),  
        ("Sakaodeuan", 50),  
        ("Pancheewa", 67),  
        ("Thaenrak", 150),  
    ]);  
    //รายชื่อที่จะทำการค้นหา  
    let names = ["Sanruethai", "Sakaodeuan", "Pancheewa", "Thaenrak", "Mether"];  
    for name in names {  
        report(&db, name);  
    }  
}
```

ค้นหารายชื่อ

แสดงผล

DETAILED CODE +LIVE DEMO

DEMO 1 - Grade_Checker

[4] output

```
===== Grade Checker =====  
Sanruethai   : 85 points [grade A]  
Sakaodeuan   : 50 points [grade D]  
Pancheewa    : 67 points [grade C]  
Thaenrak     : Warning! 150 points --> Incorrect scores  
Mether       : Error! Not found
```

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

ติดตามสถานะพัสดุ ที่แต่ละสถานะมีการรับข้อมูลต่างกัน



ORDERED



SHIPPED



OUT FOR DELIVERY



DELIVERED

DETAILED CODE + LIVE DEMO

DEMO 2 - Package_Tracker

[1] PackageStatus

```
enum PackageStatus{  
    Ordered,  
    Shipped(String),  
    OutForDelivery{  
        courier_name:String,  
        estimated_hours :u32  
    },  
    Delivered,  
}
```

Unit Variant

Tuple Variant

Struct Variant

#1 สร้าง ENUM

สร้างสถานะของพัสดุที่สามารถเป็นไปได้
โดยมี 3 VARIANT

****ต้องตั้งชื่อขึ้นต้นด้วยตัวใหญ่เท่านั้น****

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

[2] describe_status

```
fn describe_status(status: &PackageStatus) -> String {  
    match status{  
        PackageStatus::Ordered => format!("Your order has been placed."),  
        PackageStatus::Shipped(courier) => format!("Your order has been shipped via {}. ",courier),  
        PackageStatus::OutForDelivery{courier_name , estimated_hours }  
            => format!("{} is delivering your package, arriving in about {} hour(s).",courier_name,estimated_hours ),  
        PackageStatus::Delivered=> format!("Your package has been delivered!"),  
    }  
}
```

#2 match แต่ละ variant

รับสถานะพัสดุจากนั้นคืนข้อความบอกลูกค้า
ผ่านการ match แยกตามชนิดของสถานะ

Ordered

Out For Delivery

Shipped

Derivered

DETAILED CODE + LIVE DEMO

DEMO 2 - Package_Tracker

[2] describe_status

```
fn describe_status(status: &PackageStatus) -> String {  
    match status {  
        PackageStatus::Ordered => format!("Your order has been placed."),  
        PackageStatus::Shipped(courier) => format!("Your order has been shipped via {}. ", courier),  
        PackageStatus::OutForDelivery { courier_name, estimated_hours }  
            => format!("{} is delivering your package, arriving in about {} hour(s). ", courier_name, estimated_hours ),  
        PackageStatus::Delivered => format!("Your package has been delivered!"),  
    }  
}
```

ยืมข้อมูลมาดูชั่วคราว ไม่ต้องการเป็นเจ้าของ

เนื้อหาเพิ่มเติม Ownership & Borrowing

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

[2] describe_status

```
fn describe_status(status: &PackageStatus) -> String {  
    match status{  
        PackageStatus::Ordered => format!("Your order has been placed."),  
        PackageStatus::Shipped(courier) => format!("Your order has been shipped via {}. ",courier),  
        PackageStatus::OutForDelivery{courier_name , estimated_hours }  
            => format!("{}", courier_name, estimated_hours ),  
        PackageStatus::Delivered=> format!("Your package has been delivered!"),  
    }  
}
```

Unit Varaint ไม่มีค่าที่ต้องดึงออกมาใช้

- ★ `format!()` คือ macro ที่สร้างข้อความ (String) จากแม่แบบ สามารถใช้ `.to_string()` ต่อท้ายข้อความแทนก็ได้ (เพราะฟังก์ชันต้องส่งค่า String คืนกลับไป)
ข้อสังเกต: คล้ายกับ `println!()` แต่แค่ไม่ได้พิมพ์ข้อความออกมา

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

[2] describe_status

```
fn describe_status(status: &PackageStatus) -> String {  
    match status{  
        PackageStatus::Ordered => format!("Your order has been placed."),  
        PackageStatus::Shipped(courier) => format!("Your order has been shipped via {}. ", courier),  
        PackageStatus::OutForDelivery{courier_name , estimated_hours }  
            => format!("{}", courier_name, estimated_hours ),  
        PackageStatus::Delivered=> format!("Your package has been delivered!"),  
    }  
}
```

Tuple Variant ดึงชื่อคนส่งของมาใช้

จะสามารถดึงค่าในวงเล็บจาก enum ออกมาใช้งานได้ก็ต่อเมื่อใช้ match จากนั้นนำไปใส่ในข้อความ

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

[2] describe_status

```
fn describe_status(status: &PackageStatus) -> String {  
    match status{  
        PackageStatus::Ordered => format!("Your order has been placed."),  
        PackageStatus::Shipped(courier) => format!("Your order has been shipped via {}. ",courier),  
        PackageStatus::OutForDelivery{courier_name , estimated_hours }  
            => format!("{}", courier_name,estimated_hours ),  
        PackageStatus::Delivered=> format!("Your package has been delivered!"),  
    }  
}
```

Struct Variant ดึงค่าตามฟิลด์

จะสามารถดึงค่าในฟิลด์ใน enum ออกมาใช้ได้เหมือนกับ tuple variant แต่ต่างกันว่าเรียกใช้ตามชื่อ

DETAILED CODE +LIVE DEMO

DEMO 2 - Package_Tracker

[3] main()

```
fn main() {
    println!("===== Package Status Tracker =====");
    let status = vec![
        PackageStatus::Ordered,
        PackageStatus::Shipped("Kerry Express".to_string()),
        PackageStatus::OutForDelivery{
            courier_name: String::from("Peewara"),
            estimated_hours: 1
        },
        PackageStatus::Delivered,
    ];
    for s in &status {
        println!("{}", describe_status(s));
    }
    println!("===== Thank you =====");
}
```

[4] Output

```
===== Package Status Tracker =====
Your order has been placed.
Your order has been shipped via Kerry Express.
Peewara is delivering your package, arriving in about 1 hour(s).
Your package has been delivered!
===== Thank you =====
```

PART 3

RUST VS OTHER LANGUAGE & PPL ANALYSIS



RUST VS OTHER LANGUAGE

SYNTAX

-RUST

เขียนสั้น ชัดเจน รวมการสร้างตัวแปรและ
แกะค่าในตัวเดียว ด้วยMATCH หรือ IF LET

-OTHER LANGUAGE

เน้นใช้ CLASS HIERARCHY
(INTERFACE/INHERITANCE) หรือการ
ต่อยอด ENUM ตั้งแต่เริ่มร่วมกับคำสั่ง
SWITCH / MATCH-CASE (หรือใช้
STD::VARIANT ใน C++)

SEMANTICS / BEHAVIOR

-RUST

ENUM แต่ละตัวเก็บข้อมูลต่างชนิดกันได้
เลย และMATCHสามารถคืนค่าออกมาใช้ได้
ทันที

-OTHER LANGUAGE

ENUM ตั้งแต่เริ่มเน้นเก็บเฉพาะค่าคงที่
(CONSTANTS) หากต้องการเก็บข้อมูลต่าง
ชนิดกัน ต้องสร้าง CLASS แยกเป็นคลาส
ย่อย หรือใช้ไลบรารีเสริม

RUST VS OTHER LANGUAGE

TYPE SYSTEM

-RUST

ออกแบบเป็น ALGEBRAIC DATA TYPES
ตัดปัญหาเรื่อง NULL ทิ้ง แล้วแทนด้วย
OPTION

-OTHER LANGUAGE

ตรวจสอบความครอบคลุมได้ไม่สมบูรณ์
ตั้งแต่ตอน COMPILE โดย C++ / PYTHON
เสี่ยงตกหล่นในรันไทม์ ขณะที่ JAVA เพิ่ง
รองรับการตรวจแบบครอบคลุมใน SEALED
TYPES เวอร์ชันใหม่ ๆ

MEMORY MANAGEMENT

-RUST

จองหน่วยความจำแบบ TAGGED UNION
เท่าที่จำเป็น ทำงานไว ไม่มีระบบ GARBAGE
COLLECTION มาดึงเครื่อง

-OTHER LANGUAGE

ภาษาอย่าง JAVA และ PYTHON ต้องจอง
พื้นที่สร้าง OBJECT บน HEAP และพึ่งพา
GARBAGE COLLECTOR ส่วน C++ บริหาร
หน่วยความจำเองแต่เสี่ยงเรื่อง TYPE
ACCESS ERROR

RUST VS OTHER LANGUAGE

SAFETY

-RUST

COMPILER ดี แต่ต้องเขียนดักให้ครบทุกกรณี ไม่จึ้นจะ COMPILE ไม่ผ่าน

-OTHER LANGUAGE

มีโอกาสเกิด RUNTIME ERROR จากกรณีตกหล่น (UNHANDLED EXCEPTION), ข้อผิดพลาดจาก NULL หรือการเข้าถึงประเภทข้อมูลผิดประเภทในรันไทม์

RUST EXAMPLE

```
// rust
// ประกาศ enum
enum PaymentStatus {
    Pending,                // กำหนดสถานะ
    Success(u32),
    Failed(String),
}

// ประกาศฟังก์ชัน รับค่า status
fn check_payment(status: PaymentStatus) {

    // ตรวจสอบค่าของ status ว่าตรงกับสถานะไหน
    match status {
        PaymentStatus::Pending => println!("กำลังตรวจสอบชำระเงิน..."),
        PaymentStatus::Success(id) => println!("ชำระเงินสำเร็จ! รหัสสลิป: {id}"),
        PaymentStatus::Failed(reason) => println!("ชำระเงินไม่สำเร็จ: {reason}"),
    }
}

fn main() {

    let status = PaymentStatus::Success(98765);

    // เรียกใช้ฟังก์ชัน
    check_payment(status);
}
```

JAVA EXAMPLE

```
// java code

// ประกาศ interface
interface PaymentStatus {
    void check();
}

// สร้างคลาส Pending
class Pending implements PaymentStatus {

    public void check() {
        System.out.println("กำลังตรวจสอบชำระเงิน...");
    }
}

class Success implements PaymentStatus {
    private int transactionId;
    public Success(int id) { this.transactionId = id; } // คอนสตรัคเตอร์

    public void check() {
        System.out.println("ชำระเงินสำเร็จ! รหัสสลิป: " + transactionId);
    }
}

// สร้างคลาส
class Failed implements PaymentStatus {
    private String reason;
    public Failed(String reason) { this.reason = reason; } // คอนสตรัคเตอร์

    public void check() {
        System.out.println("ชำระเงินไม่สำเร็จ: " + reason);
    }
}

public class Main {

    public static void main(String[] args) {

        PaymentStatus status = new Success(98765);

        // เรียกใช้เมธอด
        status.check();
    }
}
```

C++ EXAMPLE

```
// C++
#include <iostream>
#include <string>

// ประกาศ Interface (Abstract Base Class)
class PaymentStatus {
public:
    virtual void check() const = 0; // Pure virtual function
    virtual ~PaymentStatus() = default; // Virtual destructor สำหรับ P
};

// สร้างคลาส Pending
class Pending : public PaymentStatus {
public:
    void check() const override {
        std::cout << "กำลังตรวจสอบชำระเงิน..." << std::endl;
    }
};

// สร้างคลาส Success
class Success : public PaymentStatus {
private:
    int transactionId;

public:
    // คอนสตรัคเตอร์รับค่า id มาบันทึกไว้
    Success(int id) : transactionId(id) {}

    void check() const override {
        std::cout << "ชำระเงินสำเร็จ! รหัสสลิป: " << transactionId << std::endl;
    }
};

// สร้างคลาส Failed
class Failed : public PaymentStatus {
private:
    std::string reason;

public:
    Failed(const std::string& reason) : reason(reason) {}

    void check() const override {
        std::cout << "ชำระเงินไม่สำเร็จ: " << reason << std::endl;
    }
};

int main() {
    // ใช้ Pointer ของ Class แม่ (Interface) เพื่อเรียกใช้ Polymorphism
    PaymentStatus* status = new Success(98765);

    // เรียกใช้เมธอด
    status->check();

    // คืนหน่วยกิต
    delete status;

    return 0;
}
```

PYTHON EXAMPLE

```
#python
from abc import ABC, abstractmethod

# ประกาศ Interface (Abstract Base Class)
class PaymentStatus(ABC):

    @abstractmethod
    def check(self):
        pass

# สร้างคลาส Pending
class Pending(PaymentStatus):

    def check(self):
        print("กำลังตรวจสอบชำระเงิน...")

# สร้างคลาส Success
class Success(PaymentStatus):

    def __init__(self, transaction_id: int):
        self.transaction_id = transaction_id # บันทึกค่า id ไว้

    def check(self):
        print(f"ชำระเงินสำเร็จ! รหัสสลิป: {self.transaction_id}")

# สร้างคลาส Failed
class Failed(PaymentStatus):

    def __init__(self, reason: str):
        self.reason = reason # บันทึกข้อความสาเหตุไว้

    def check(self):
        print(f"ชำระเงินไม่สำเร็จ: {self.reason}")

# การใช้งานในฟังก์ชันหลัก
if __name__ == "__main__":
    status: PaymentStatus = Success(98765)

    # เรียกใช้เมธอด
    status.check()
```

ANALYSIS

ความแตกต่างสำคัญระหว่าง RUST กับ ภาษาอื่น (C++, JAVA, PYTHON)

คือการเปลี่ยนวิธีการจัดการสถานะและข้อมูลหลากหลายรูปแบบ จากเดิมที่ภาษาอื่นใช้แนวคิด OOP / DYNAMIC TYPING (การใช้ CLASS HIERARCHY, DYNAMIC DISPATCH หรือ STD::VARIANT) มาเป็นการใช้ ALGEBRAIC DATA TYPES (SUM TYPES) ผ่าน ENUM ที่แนบข้อมูลไว้ใน VARIANT ได้โดยตรง ร่วมกับระบบ EXHAUSTIVE PATTERN MATCHING (MATCH) และการจัดเก็บแบบ TAGGED UNION บน STACK ทำให้ RUST สามารถจัดการข้อมูลได้กะทัดรัด ตรวจสอบกรณีต่างๆ ได้อย่างรวดเร็ว และไม่มี OVERHEAD จาก GARBAGE COLLECTOR (GC) หรือการจอง HEAP โดยไม่จำเป็น

เหตุผลด้านการออกแบบภาษา

RUST ถูกออกแบบมาบนปรัชญา "ZERO-COST ABSTRACTIONS, COMPILE-TIME SAFETY และ DATA-ORIENTED DESIGN" โดยมุ่งเน้นย้ายข้อผิดพลาดทั้งหมดที่อาจเกิดขึ้นขณะทำงาน (RUNTIME ERRORS เช่น การลืมนับค่าบางกรณี, NULL POINTER หรือ TYPE ERROR) มาให้ COMPILER บังคับตรวจจับให้ครบถ้วนตั้งแต่ขั้นตอน COMPILE TIME ภาษาจึงเลือกใช้ SUM TYPES และ PATTERN MATCHING เพื่อให้ได้ความปลอดภัยสูงสุดโดยไม่ต้องแลกมาด้วย GARBAGE COLLECTOR เหมือน JAVA/PYTHON และยังคงประสิทธิภาพระดับ SYSTEM-LEVEL PROGRAMMING เท่า C/C++

PPL-PERSPECTIVE

SYNTAX

ประกาศ ENUM เพื่อรวมกลุ่มของข้อมูลหลายแบบไว้ด้วยกันและใช้ไวยากรณ์ MATCH หรือ IF LET เพื่อทำการเช็คเงื่อนไขและดึงข้อมูลข้างในออกมาใช้งานในบรรทัดเดียวกันได้เลย

SEMANTICS

ENUM ของ RUST แต่ละตัว สามารถผูกข้อมูลต่างชนิดกันไว้ข้างในได้ และตัว MATCH ใน RUST มีพฤติกรรมเป็น EXPRESSION คือสามารถประมวลผลแล้วคืนค่า ออกมาใส่ตัวแปรต่อได้ทันที

TYPE SYSTEM

ENUM ถูกจัดเป็น SUM TYPE คือชนิดข้อมูลที่เลือกได้แค่อย่างเดียวกจากตัวเลือกที่มี ต่างจาก STRUCT ที่เป็น PRODUCT TYPE คือรวมทุกอย่างพร้อมกัน เมื่อนำ ENUM มาผสมกับ GENERICS (การใส่ชนิดข้อมูลแบบยืดหยุ่นได้ เช่นใน `OPTION<T>` และ `RESULT<T, E>`) ตัวคอมไพเลอร์จะบังคับให้เราต้องจัดการทั้งกรณี "สำเร็จ" และ "ผิดพลาด" ตั้งแต่ตอนเขียนโค้ด นอกจากนี้ RUST นำ `OPTION<T>` มาแทนที่การใช้ `NULL` ทำให้ไม่มีปัญหา ERROR ที่มาจากการเรียกใช้ค่าที่ไม่มีอยู่จริง

PPL-PERSPECTIVE

MEMORY AND RESOURCE MANAGEMENT

เบื้องหลังการทำงาน ENUM จะถูกจัดเก็บ
แบบ TAGGED UNION ใน MEMORY โดย
จองพื้นที่เท่ากับขนาดของตัวที่ใหญ่ที่สุด บวก
กับ TAG เล็กๆ ไว้ระบุประเภท

ABSTRACTION/OTHER PPL CONCEPT

ENUM ของ RUST แต่ละตัว สามารถผูก
ข้อมูลต่างชนิดกันไว้ข้างในได้ และตัว
MATCH ใน RUST มีพฤติกรรมเป็น
EXPRESSION คือสามารถประมวลผลแล้ว
คืนค่า ออกมาใส่ตัวแปรต่อได้ทันที

WHY RUST?

เขียนง่ายเหมือนภาษา HIGH-LEVEL แต่
คอมไพล์ออกมาเป็น MACHINE CODE ที่
ทำงานเร็วพอกับ C/C++ โดยไม่มีระบบ
GARBAGE COLLECTION มาดึงเครื่องให้
ช้าลง อีกทั้งยังตรวจจับข้อผิดพลาดได้ตั้งแต่
COMPILE TIME เป็นส่วนใหญ่ ทำให้โค้ด
ปลอดภัยและมีประสิทธิภาพสูง โดยไม่มี
RUNTIME OVERHEAD เพิ่มขึ้น

PART 4



EXERCISES & COMMON MISTAKES & CHALLENGE

COMMON MISTAKE

Mistake 1 - Non exhaustive match

✗ INCORRECT

```
enum Payment {
    Cash,
    CreditCard,
    PromptPay,
}

fn main() {
    let payment = Payment::Cash;

    match payment {
        Payment::Cash => println!("ชำระด้วยเงินสด"),
    }
}
```

✓ CORRECT

```
enum Payment {
    Cash,
    CreditCard,
    PromptPay,
}

fn main() {
    let payment = Payment::Cash;

    match payment {
        Payment::Cash => println!("ชำระด้วยเงินสด"),
        _ => println!("ชำระด้วยวิธีอื่น"),
    }
}
```

Why?

Payment มี variants 3 อัน คือ Cash , CreditCard , PromptPay แต่ match จัดการเพียง Cash จึงทำให้เกิด error non-exhaustive เพราะ match ใน Rust ต้องครอบคลุมทุก variant ของ enum ดังนั้นใน Correct Code จึงใช้ _ เพื่อครอบคลุมทุกกรณีที่เหลือให้แสดงผลออกมาเป็น "ชำระด้วยวิธีอื่น"

COMMON MISTAKE

Mistake 2 - สับสนเรื่อง Ownership เมื่อใช้ match

✗ INCORRECT

```
enum Message {
    Write(String),
}
fn main() {
    let msg = Message::Write(String::from("have a nice day"));
    match msg {
        Message::Write(text) => println!("{}", text),
    }
    match msg {
        Message::Write(text) => println!("{}", text),
    }
}
```

✓ CORRECT

```
enum Message {
    Write(String),
}
fn main() {
    let msg = Message::Write(String::from("have a nice day"));
    match &msg {
        Message::Write(text) => println!("{}", text),
    }
    match &msg {
        Message::Write(text) => println!("{}", text),
    }
}
```

Why?

match msg จะเอาค่าที่อยู่ใน msg มาใช้ใน match ถ้าข้อมูลนั้นเป็น String อาจทำให้ค่าถูกย้ายออกไปแล้วทำให้ msg ใช้ต่อไม่ได้ แต่การใช้ match &msg จะเป็นการยืมข้อมูลมาดู ทำให้ยังสามารถใช้ msg ต่อได้หลังจาก match เนื้อหาที่เกี่ยวข้อง : Ownership & Borrowing

COMMON MISTAKE

Mistake 3 - ลำดับ Pattern ใน match ผิด

✗ INCORRECT

```
enum Status {
    Success,
    Error,
    Pending,
}

fn show_status(status: Status) {
    match status {
        Status::Success => println!("สถานะสำเร็จ"),
        _ => println!("สถานะอื่น"),
        Status::Error => println!("เกิดข้อผิดพลาด"),
    }
}

fn main() {
    show_status(Status::Error);
}
```

✓ CORRECT

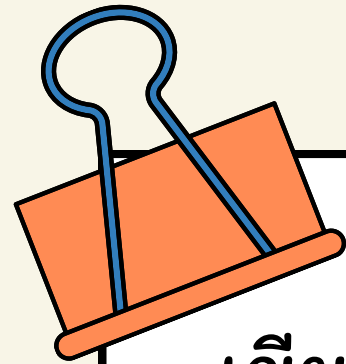
```
enum Status {
    Success,
    Error,
    Pending,
}

fn show_status(status: Status) {
    match status {
        Status::Success => println!("สถานะสำเร็จ"),
        Status::Error => println!("เกิดข้อผิดพลาด"),
        _ => println!("สถานะอื่น"),
    }
}

fn main() {
    show_status(Status::Error);
}
```

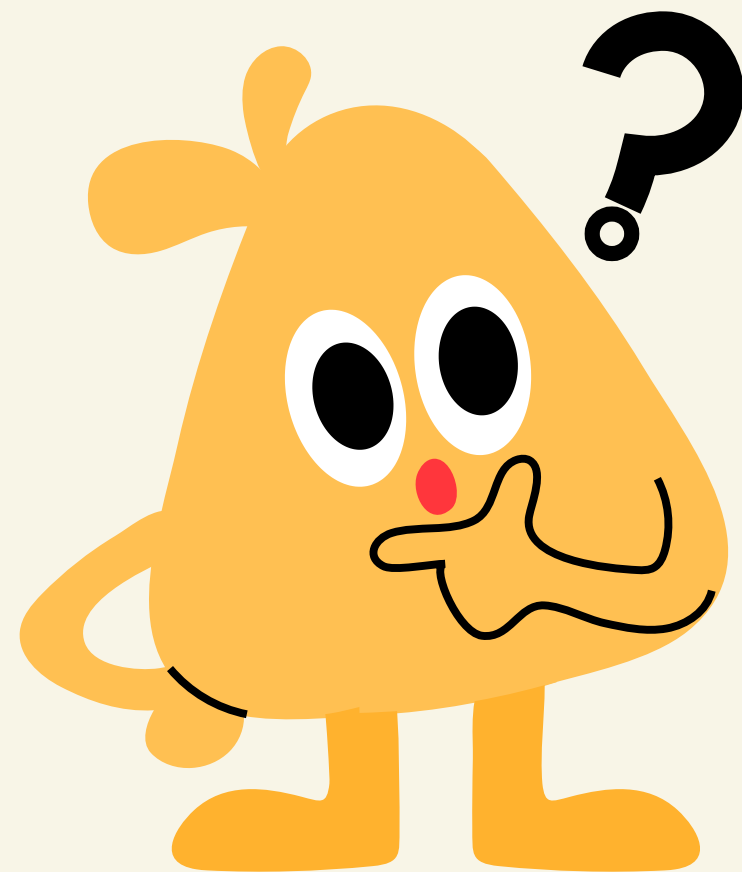
Why?

match จะตรวจสอบ pattern จากบนลงล่าง ใน Incorrect Code _ หมายถึงทุกกรณีที่เหลือ จึงทำให้ Status::Error ที่อยู่ด้านหลังไม่สามารถทำงานได้ ดังนั้นควรวาง pattern ที่เฉพาะเจาะจงไว้ก่อน แล้วค่อยใช้ _ สำหรับกรณีที่เหลือ



EXERCISE — CALCULATE SHAPE AREA

เขียน enum ชื่อ Shape มี variant 3 ตัว ได้แก่ Circle, Rectangle, Triangle
จากนั้นเขียนฟังก์ชัน area ที่รับ Shape และคำนวณพื้นที่แต่ละรูปโดยใช้ match



HINT

พื้นที่วงกลม = $\pi \times R^2$

พื้นที่สี่เหลี่ยม = กว้าง $\times$ ยาว

พื้นที่สามเหลี่ยม = $1/2 \times$ ฐาน $\times$ สูง

Solution

```
enum Shape {
    Circle(f64),    //tuple-like
    Rectangle(f64, f64), //tuple-like
    Triangle { base: f64, height: f64 }, //struct-like
}

fn area(shape: &Shape) -> f64 {
    match shape {
        Shape::Circle(r) => std::f64::consts::PI * r * r,
        Shape::Rectangle(w, h) => w * h,
        Shape::Triangle { base, height } => 0.5 * base * height,
    }
}

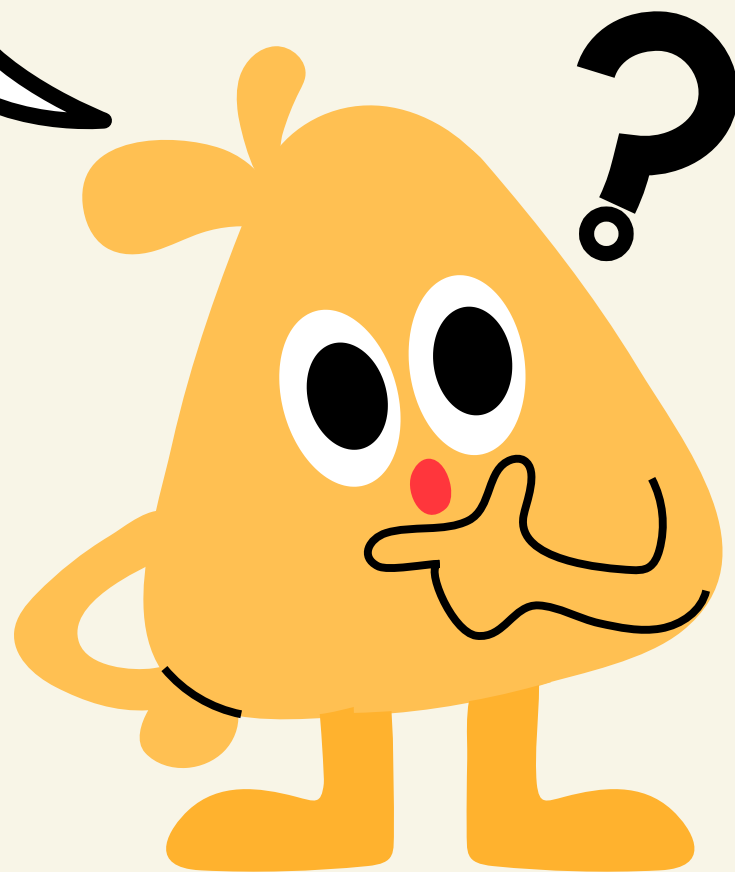
fn main() {
    let circle = Shape::Circle(2.0);
    let rectangle = Shape::Rectangle(3.0, 4.0);
    let triangle = Shape::Triangle { base: 5.0, height: 6.0 };
    println!("Circle = {:.2}", area(&circle)); //แสดงผลลัพธ์เป็นทศนิยม 2 ตำแหน่ง
    println!("Rectangle = {:.2}", area(&rectangle));
    println!("Triangle = {:.2}", area(&triangle));
}
```



CHALLENGE — คำถามท้าทายผู้ฟัง



Code นี้ compile
ผ่านหรือไม่?
และถ้าไม่ผ่านผิด
ตรงไหน?



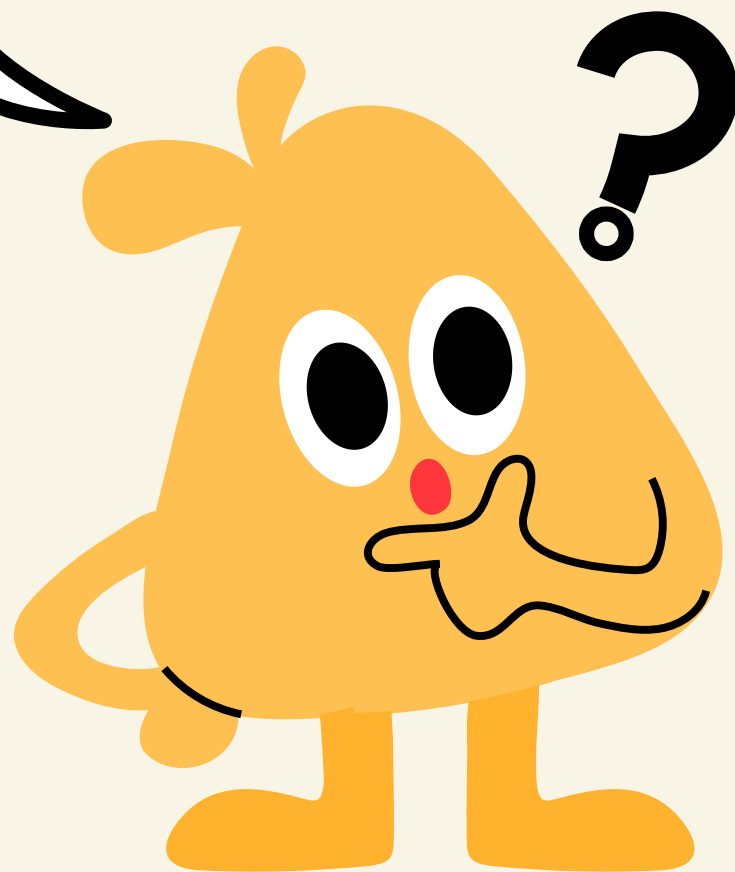
```
enum BookStatus {
    Available,
    Borrowed(String, u32),
    Reserved { name: String, days: u32 },
}

fn check_book(status: &BookStatus) {
    match status {
        BookStatus::Borrowed(name, days) if *days > 30 => {
            println!("{}", borrowed for more than 30 days", name);
        }
        BookStatus::Borrowed(name, days) => {
            println!("{}", borrowed for {} days", name, days);
        }
        BookStatus::Available => {
            println!("The book is available");
        }
    }
    if let BookStatus::Reserved { name, days } = status {
        if *days > 3 {
            println!("{}", has reserved the book for more than 3 days", name);
        }
    }
}
```

CHALLENGE — คำถามท้าทายผู้ฟัง



Code นี้ compile
ผ่านหรือไม่?
และถ้าไม่ผ่านผิด
ตรงไหน?



```
enum BookStatus {  
    Available,  
    Borrowed(String, u32),  
    Reserved { name: String, days: u32 },  
}  
  
fn check_book(status: &BookStatus) {  
    match status {  
        BookStatus::Borrowed(name, days) if *days > 30 => {  
            println!("{}", borrowed for more than 30 days", name);  
        }  
        BookStatus::Borrowed(name, days) => {  
            println!("{}", borrowed for {} days", name, days);  
        }  
        BookStatus::Available => {  
            println!("The book is available");  
        }  
    }  
  
    if let BookStatus::Reserved { name, days } = status {  
        if *days > 3 {  
            println!("{}", has reserved the book for more than 3 days", name);  
        }  
    }  
}
```

Solution

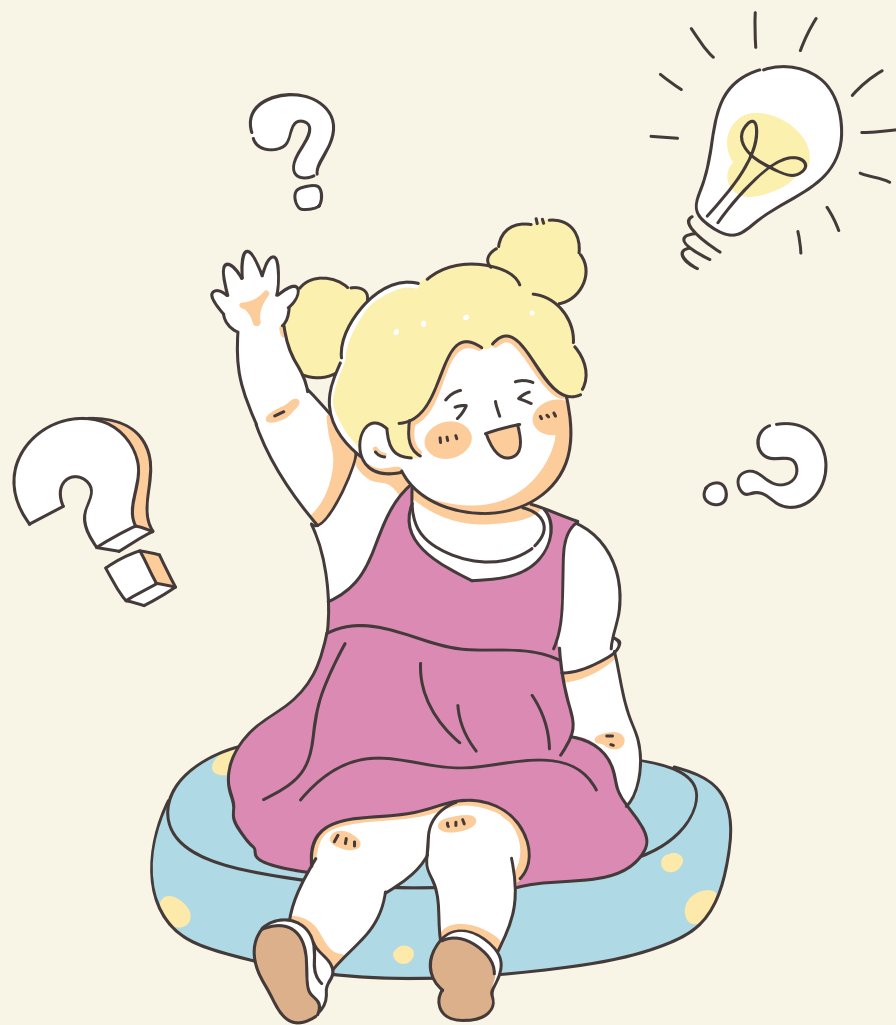
```
error[E0004]: non-exhaustive patterns: `&BookStatus::Reserved { .. }` not covered
--> src/main.rs:7:11
7 |         match status {
  |         ^^^^^^ pattern `&BookStatus::Reserved { .. }` not covered
```

✗ Compile ไม่ผ่าน

- match ยังไม่ครอบคลุม BookStatus::Reserved
- if let ตรวจสอบ Reserved แต่ ไม่ถือว่าเป็นการครอบคลุมของ match

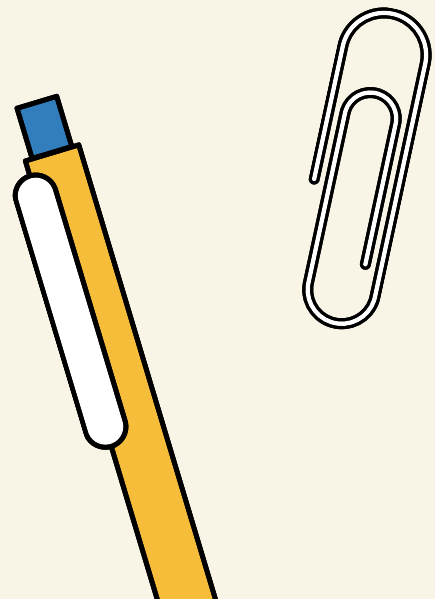


Correct Code

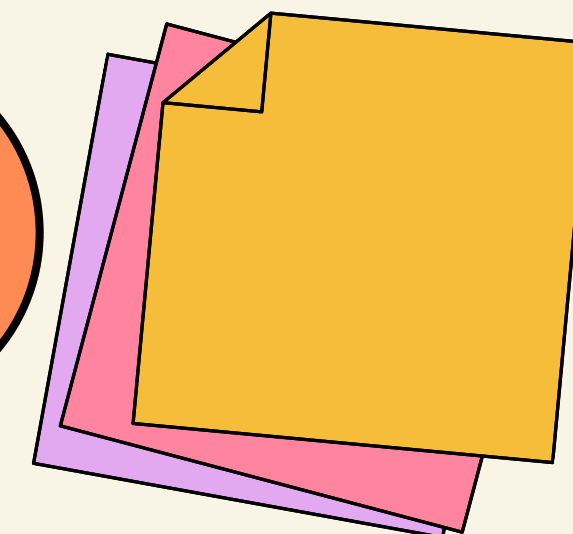


```
enum BookStatus {
    Available,
    Borrowed(String, u32),
    Reserved { name: String, days: u32 },
}

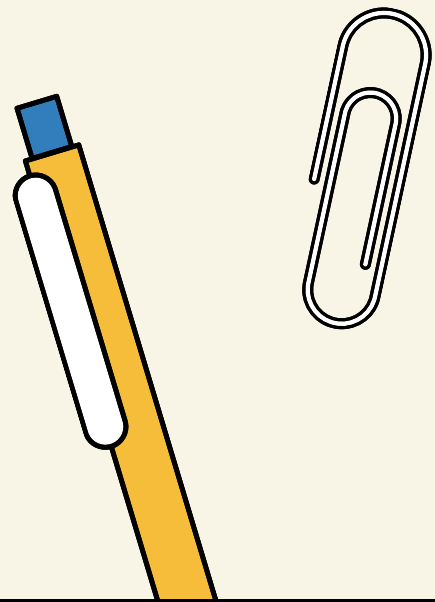
fn check_book(status: &BookStatus) {
    match status {
        BookStatus::Borrowed(name, days) if *days > 30 => {
            println!("{}", borrowed for more than 30 days", name);
        }
        BookStatus::Borrowed(name, days) => {
            println!("{}", borrowed for {} days", name, days);
        }
        BookStatus::Available => {
            println!("The book is available");
        }
        BookStatus::Reserved { .. } => {
            println!("The book is reserved");
        }
    }
    if let BookStatus::Reserved { name, days } = status {
        if *days > 3 {
            println!("{}", has reserved the book for more than 3 days", name);
        }
    }
}
```



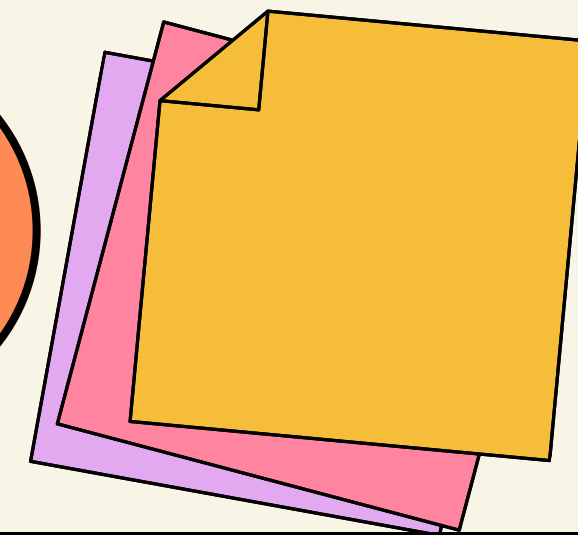
KEYTAKEAWAYS-สรุปสิ่งที่ควรจำ



1. Pattern Matching ใช้ match หรือ if let เพื่อตรวจสอบ Variant และดึงข้อมูลออกมาใช้งาน
2. match ต้องครอบคลุมทุกกรณี (Exhaustiveness) ถ้าไม่ครบจะ Compiler ไม่ได้
3. if let : เหมาะสำหรับกรณีที่สนใจเพียง Variant เดียว แทนการ match ทุกตัว
4. Pattern ใน match ตรวจสอบบนลงล่าง จึงควรวาง Pattern ที่เฉพาะเจาะจงก่อน แล้วค่อยวางกรณีทั่วไป
หรือ '_' (Wildcard) ไว้ล่างสุด
5. Option<T> ใช้ตรวจสอบว่าข้อมูลนั้นมีค่า(Some) หรือ ไม่มีค่า (None) แทนการใช้ null
6. Result<T, E> ใช้จัดการผลลัพธ์ที่อาจสำเร็จ (Ok) หรือเกิดข้อผิดพลาด (Error)
7. Ownership & Borrowing มีผลต่อการใช้ match โดยเฉพาะเมื่อ Enum เก็บข้อมูลที่มี Ownership เช่น String



KEYTAKEAWAYS-สรุปสิ่งที่ควรจำ



ความเชื่อมโยงกับหัวข้ออื่น ๆ ในวิชา PPL

Type Safety & Type Systems

บังคับให้จัดการข้อผิดพลาดตั้งแต่ Compile time ส่งผลให้มีความปลอดภัยและได้ประสิทธิภาพสูง

Control Flow

Pattern Matching ใน Control Structure ตรวจสอบทั้งประเภทและดึงข้อมูลภายในขั้นตอนเดียว

Error Handling Paradigm

จัดการข้อผิดพลาดแบบคืนค่าข้อมูลแทนการโยน Exception ช่วยบังคับให้ตรวจสอบแล้วเปลี่ยนลำดับการทำงาน

The Null Pointer Problem

ภาษาทราบได้อย่างชัดเจนว่าค่าใดที่ไม่มีค่าและให้ผู้สร้างจัดการกับกรณีนั้นเสมอ

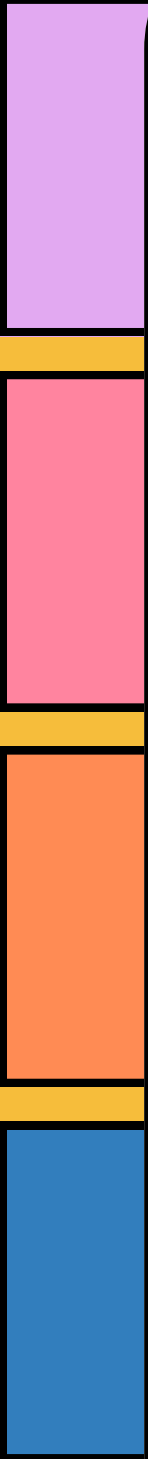


REFERENCES—เอกสารอ้างอิง

1. [The Rust Programming Language — Rust Book]
2. [Rust by Example / Rust Reference]
3. [Rust Documentation]
4. [W3Schools]
5. [Workshop EDM - Rust 2024]

GitHubRepository :

<https://github.com/670710143/rust-tutorial-2569>



QUESTIONS & ANSWERS

กลุ่มที่ 9 - Enums & Pattern Matching